```
class CPUBoundedSpec ... {
  ...

  implicit val ec =
    ExecutionContext.fromExecutor(Executors.newFixedThreadPool(12))     ❶

  "calling to a local function async" should {
    "eventually return 3 times faster" in {

      val result: Future[Done] =
        Source(1 to 10)
          .mapAsync(10)(each => Future(parsingDoc(each.toString)))      ❷
          .run()

      Await.result(result, 10.seconds)
    }
  }
  ...
}
```

❶ Number of available threads

❷ Sets parallelism and wraps it in a future

We ran this on a laptop with 6 cores and 12 processes available for multithreading; hence the setting `Executors.newFixedThreadPool(12)`. This way, the function can run in a different thread than the caller (that is, the stream).

To run this test, set the previous test back to `ignore` and this last one to `in`, and then enter the following:

```
sbt:code> chapter16/testOnly *CPUBoundedSpec
```

This test has much better performance:

```
6.012s
6.064s
6.085s
6.093s
```